

Migrating GEOUNED to pyOCC

Technical report on the CAD backend refactor

GEOUNED project — branch `georeverse-migration`

September 6, 2026

Project	GEOUNED-org/GEOUNED
Branch	<code>georeverse-migration</code>
Default engine	<code>ocp</code> (OCP / pybind11)
Supported engines	<code>freecad · occ · ocp</code>
Scope	CadToCsg (GEOUNED) + CsgToCad (GEORreverse)
Source	working log <code>CLAUDE.md</code>

3 / 3 CAD engines with the test suite green 0 lost particles (<code>test_models</code> corpus, 143 dlsuned runs) 93.6% tallies within 2σ / 0% above 3σ 0 direct <code>Part/FreeCAD</code> imports in GEOUNED
--

Contents

1	Executive summary	1
2	Context and motivation	2
2.1	The problem that triggered the migration	2
2.2	Decision: <code>pythonocc-core</code> , not <code>CadQuery</code>	2
3	Architecture: the <code>geo</code> package	2
3.1	Attempt 1 (retired): the <code>GeometryBackend ABC</code>	2
3.2	Attempt 2 (current): the <code>geo</code> package	2
3.3	Design of the <code>geo</code> classes	3
4	Decoupling FreeCAD from GEOUNED	3
4.1	Unifying duplicated representations	4
4.2	Two surface-numbering scopes	4
5	Writer reorganisation	4
6	Verification campaigns	5
6.1	Point-by-point <code>check_sign</code> verification	5
6.2	MCNP stochastic volume check	5
7	Geometry fixes	5
8	Phased pyOCC migration	7
8.1	Third engine: <code>OCP</code> (<code>pybind11</code>)	7
9	GEORreverse migration	8
10	CAD defect detection and repair	9

11 Current status and open work	10
11.1 Consolidated status	10
11.2 Open	10
12 Appendix: environment and tooling	10

1 Executive summary

GEOUNED converts CAD geometry into CSG geometry (and back) for the Monte Carlo transport codes **MCNP**, **OpenMC**, **PHITS** and **Serpent**. It historically depended entirely on FreeCAD’s `Part` API and, in particular, on `BOPTools.SplitAPI.slice` for solid decomposition. That coupling has been removed.

The work done falls into five blocks:

- **Kernel isolation.** All code that touches FreeCAD/OCCT now lives in a single package, `geo`, behind a stable name-level interface. No other file in GEOUNED imports `Part` or `FreeCAD`.
- **Three geometry engines.** `geo` has three interchangeable implementations selected via the environment variable `GEOUNED_CAD_ENGINE`: FreeCAD (`freecad`), pythonocc-core / SWIG (`occ`) and OCP / pybind11 (`ocp`, currently the default).
- **Quantitative verification.** Every fix has been validated by two independent methods: point-by-point checking of `check_sign` against independently built CAD geometry, and a stochastic volume check that runs the resulting MCNP model in `dlsuned` (tally ≈ 1.0).
- **Data-driven geometry debugging.** Dozens of real bugs — many predating the migration — found and fixed by following concrete traces: AND/OR signs in composite surfaces, tangency degeneracies in `Gsplit`, surface classification, tolerance scaling and residual *slivers*.
- **CAD defect repair.** A new load-time layer that detects corrupt solids (pathologically short edges, duplicated boundary rings, phantom cuts) and repairs them automatically when that is fast and reliable, or flags them untranslated when it is not.

The reverse pipeline, **GEORreverse** (`CsgToCad`), which was initially out of scope because of its hard dependency on the `.FCStd` format, has received its own migration on the current branch: it is engine-switchable, exports real STEP via XCAF under the pyOCC engines, and colours each solid by material.

Status as of this report: all three engines pass their suites; the reference corpus `Solidos/test_models` translates with no lost particles and no failures above 3σ ; `occ` and `ocp` produce identical results over the whole corpus. Several GEORreverse items and a few exotic surfaces remain unported (Section 11).

2 Context and motivation

2.1 The problem that triggered the migration

FreeCAD’s boolean `Cut` can **silently** fail to split a solid into two when the cutting plane’s intersection coincides exactly with a pre-existing tangency line (for example, a plane cutting the corner where a cylinder is already tangent to two cube faces). Symptom: `Cut` returns the original, uncut solid instead of raising an error. Root cause: the resulting fragments only touch along a line (zero-area contact), producing a non-manifold shape that OCC’s BOP algorithm does not cleanly resolve. Round-tripping through a STEP export/import “fixes” it because the STEP healer re-splits the compound.

The proper fix — building a face-adjacency graph that excludes edges shared by more than two faces and reconstructing one solid per connected component — requires fine-grained topological control that FreeCAD’s `Part` API, itself a wrapper over OCCT, does not expose conveniently.

2.2 Decision: `pythonocc-core`, not `CadQuery`

- Fine-grained topological control is needed (face/edge adjacency graphs, `BRepCheck_Analyzer`, `ShapeFix_Shape`, tolerance control over `BRepAlgoAPI_*` / `BOPAlgo_Splitter`) for exactly the degenerate cases above.
- Migrating to `pyOCC` *removes* a layer (the `Part` wrapper) rather than replacing it with a different one (`CadQuery`).
- `CadQuery` is a better fit for high-level parametric construction, not the low-level topology surgery `GEOUNED`’s decomposition requires.

3 Architecture: the `geo` package

3.1 Attempt 1 (retired): the `GeometryBackend` ABC

The first design was a classic Adapter / Ports & Adapters pattern: an abstract base class `GeometryBackend` with neutral dataclasses (`GSolid` / `GFace` / `GEdge` / `GVector` / ...) that each carried a `native` object plus a reference to the backend that produced them, and a concrete `FreeCADBackend` implementation. All of `GEOUNED` called through a single injected instance (`_backend.split(gsolid, tool, tol), ...`). It was fully implemented and validated against the whole suite, but was judged **too heavy and indirect** — the dataclasses carrying their own backend and the `_backend.method(x, ...)` convention — for what it bought. It was deleted entirely along with its tests.

3.2 Attempt 2 (current): the `geo` package

`src/geouned/geo/` is the **only place in `GEOUNED` allowed to import `Part` / `FreeCAD` / `BOPTools`**.

File	Responsibility
<code>vector_geometry.py</code>	Pure math with no FreeCAD dependency: <code>GVector</code> , <code>GMatrix</code> , <code>GBoundingBox</code> and converters.
<code>surface_geometry.py</code>	Analytic predicates over surface descriptors (<code>is_same_*_surface</code> , <code>is_inside_*</code> , <code>find_can_plane</code> , plane-plane and line-line intersections).
<code>solid_defects.py</code>	Whole-solid CAD defect detection (<code>find_short_edges</code> , <code>find_split_ring_faces</code> , <code>near_surface_pair</code>).
<code>io_utils.py</code>	Process/loading utilities: <code>GLabelNode</code> (STEP label tree), <code>suppress_native_stdout</code> .
<code>_freecad_impl.py</code>	FreeCAD implementation: surface/curve classification, eagerly built topology classes (<code>GEdge</code> / <code>GWire</code> / <code>GFace</code> / <code>GShell</code> / <code>GSolid</code>) carrying real behaviour as methods, <code>Gmake_*</code> constructors and <code>Gcut</code> / <code>Gcommon</code> / <code>Gfuse</code> / <code>Gsplit</code> operations.
<code>_occ_impl.py</code>	The same name surface against <code>pythonocc-core</code> (SWIG).
<code>_ocp_impl.py</code>	The same name surface against OCP (<code>pybind11</code>).

File	Responsibility
<code>__init__.py</code>	Single import point. Reads <code>GEOUNED_CAD_ENGINE</code> once and re-exports the names from the resolved implementation.

Interchangeability principle. Engine selection happens at import time, not through dependency injection. A new engine only has to define the same class and function names; `geo/__init__.py` chooses which to re-export and the rest of `GEOUNED` does not change. `GEOReverse` replicates exactly the same pattern with its own `GEOReverse/Modules/__init__.py`.

3.3 Design of the geo classes

- Faces are eagerly classified into one of 5 analytic surfaces (plane / cylinder / cone / sphere / torus) via `Gclassify_surface`. Composite surfaces (`RoundCorner`, `Can`, `TCone`, `MultiPlane`, `MultiRoundCorner`, `ReversedConeCylinder`) are assembled by `GEOUNED` itself with `Gcut/Gcommon/Gfuse`; `geo` never models them.
- Curve classification is tolerant: it returns `None` for a degenerate edge or a real-but-unsupported type (e.g. hyperbola) rather than raising, because solids are built eagerly and an incidental edge must not abort the whole solid.
- `Gsplit()` returns a `SplitResult` that can never come back empty silently: tangency/degeneracy cases are resolved internally (tolerance retry, non-manifold reconstruction, component separation) and reported with `degenerate_case_handled=True`.
- `utils/geometry_gu.py::FaceGu / SolidGu` — a second native wrapper predating `geo`, used by the decomposition code — now **inherit** from `GFace / GSolid` and only layer the decomposition-side extras on top, instead of duplicating the construction.

4 Decoupling FreeCAD from GEOUNED

Closing the `import Part / import FreeCAD` statements was done file by file, tracing every real consumer before touching the source. `core.py` (specifically `_set_geometry_bounding_box`) was the last holdout, precisely because its `BoundingBox` is consumed outside the migration’s original scope (`void.py`, `write_files.py`). After it was closed, **zero files** in the `GEOUNED` subpackage import `Part/FreeCAD/BOPTools` directly.

4.1 Unifying duplicated representations

“A plane” had three parallel representations: the descriptor `geo.GPlane`, the `PlaneParams` class in `basic_functions_part1.py`, and the `Plane` class in `build_region/Objects.py`. They were consolidated:

- “analytic surface + bounding box → finite clipped solid” for plane/cylinder/cone was unified into three pure functions in `build_region/Objects.py` (`plane_polygon_from_box`, `cylinder_from_box`, `cone_from_box`), in `GVector/GBoundingBox` space, with no duplication.
- The `Plane/Cylinder/Cone/Sphere` classes in `Objects.py` collapsed into a single `CellSurface` wrapping a `geo` descriptor; `get_surface()` now builds `GPlane/GCylinder/...` directly via `.from_values(...)`, with no translation step and no native detour.
- The Tier-1 `*OnlyParams` classes now store `GVector` directly instead of `FreeCAD.Vector`; `check_sign_primitive` collapsed to a dict dispatch against the shared `is_inside_*` functions.

Recorded methodological lesson. A static “who reads this field” survey is necessary but not sufficient when the value passes through an intermediate object before reaching the operation that cares about its type. Several times, a **grep**-based survey concluded that every consumer tolerated **GVector**, and the real test suite surfaced native call sites the **grep** missed. Rule: follow every real **AttributeError/TypeError** from the full suite, not just static tracing.

4.2 Two surface-numbering scopes

Every **GeounedSurface** has **.bVar** (the signed id it is referred to by) and, for composite types, **.region** (the AND/OR boolean definition) and **.components** (the numbering $\leftrightarrow$ surface relation). They are assigned in two phases with genuinely different numbering: in **decomposition** the **.bVar** is a throwaway local id whose only job is to keep the AND/OR expression consistent for one CAD build; in **conversion** it is overwritten with the global, deduplicated canonical id. The two phases *cannot* read each other’s **.region**, but they do share the AND/OR **rule**, extracted into pure functions (**can_region**, **tcone_region**, **round_corner_region**, ...).

5 Writer reorganisation

The four output-format writers had near-identical implementations of several methods. They were reorganised along the real card-convention boundary:

- **write/mcnp_like/** — MCNP, Serpent and PHITS (the three MCNP-lineage text formats, sharing full-line/inline comment conventions and banners).
- **write/openmc/** — structurally different (XML/Python output, no comment-card concept).
- **write/functions.py** — shared home for **get_cell_surf_summary**, **simplify_planes**, **sorted_surfaces**, used by all four formats.

The **CommonInputWriter** mixin (inherited by **McnpInput/SerpentInput/PhitsInput**) collects what is duplicated only among the three MCNP-lineage formats, parametrised by class attributes (**_surface_formatter**, **inline_comment_char**, ...).

A real correction, confirmed with the user. The **sorted_surfaces** unification fixes an inconsistency: only MCNP applied **Surfaces.IndexOffset** when relabelling each **bVar**; Serpent/OpenMC/PHITS skipped it. All four formats now use the MCNP version. The effect is currently dormant (**IndexOffset** is 0 unless **settings.startSurf** is wired, which was also done in this migration) but it is no longer a divergence between formats.

6 Verification campaigns

6.1 Point-by-point **check_sign** verification

For every composite surface found in a model, an *independent* CAD reference solid is built via the same path **Gsplit** itself uses to build that composite, random points are sampled strictly inside the construction box, and **check_sign(point, surface)** is compared against **gsolid.is_inside(point)**.

Composite type	Result	Notes
RoundCorner	300/300	Many instances across dozens of files.
Can (Fwd / Rev)	300/300	A sampling artefact (bounding-box padding) was identified and fixed: sample inside the construction box, not the resulting solid’s box.

Composite type	Result	Notes
TCone (Fwd / Rev)	300/300	Surfaced a real bug in <code>TCone_region</code> (it ignored <code>p1_configuration/p2_configuration</code>), fixed.
MultiRoundCorner	300/300	17 instances across 13 files. First end-to-end validation of this type.
MultiPlane	300/300	Surfaced that <code>multiplane()</code> could never detect a MultiPlane (instance-vs-class comparison), fixed.
ReversedConeCylinder	partial	Structurally unreachable from the decomposition scan; verified from the conversion side with one real discrepancy on <code>hylife-v06</code> solid113 (271/300), tied to a known GEORreverse mismatch.

6.2 MCNP stochastic volume check

A different, more end-to-end question: does a *fully translated* MCNP model, actually run, reproduce the **same volumes** as the original CAD solids? The model is exported with `volSDEF=True` (isotropic photon source from an enclosing sphere + an F4 track-length tally on each cell, normalised by the CAD-computed volume), run in `MODE P + VOID` in `dlsuned`, and the expected tally result is exactly **1.0**. A significant deviation ($> 3\sigma$) means the written surface signs do not reconstruct the same solid.

Corpus Solidos/test_models (excluding Big_*)	Value
Files that translate	133 / 138
Tallies within 2σ of 1.0	93.6%
Marginal tallies ($2-3\sigma$, expected noise)	6.4%
Real failures ($> 3\sigma$)	0%
Files with lost particles	0 / 143 runs
occ vs ocp divergence over the corpus	byte-for-byte identical

The 5 files that do not translate are genuine CAD defects that the detection layer (Section 10) correctly flags, plus one native-crash case accepted as a limitation (`ConeSphere.stp` under `occ/ocp`).

7 Geometry fixes

Dozens of real bugs, most predating the migration (confirmed against the last pre-migration commit), found by following concrete traces — never by guessing — and verified before and after. Grouped by nature:

Category	Representative examples	Impact
AND/OR signs in composite surfaces	<code>can_region</code> cone + apex-plane Reversed branch used AND where the physical sign requires OR (two instances); <code>TCone_region</code> substituted a single orientation-based value for the real configuration; <code>round_corner_region</code> applied <code>fwd_cyl</code> twice in the <code>p1 == p2</code> branch; <code>cone_apex_plane</code> double-compensated the Reversed case; <code>add_reversedCC</code> redesigned (<code>plane_region AND (s1 AND ... AND sn)</code>) and its <code>AddPlanes</code> moved from unconditional OR to AND; <code>AdjacentMultiplanePlanes</code> became a list-of-lists (AND within a group, OR across groups).	Wrong CSG volume with a valid boolean expression; in several cases “leaks” that engulfed half the model. Verified with <code>dlsuned</code> after each fix.

Category	Representative examples	Impact
Degeneracy / tangency in Gsplit	Coaxial cone-cone split fallback (<code>_try_coaxial_cone_split</code>): analytic reconstruction of the exact intersection circle, pre-split of that edge and a retry with fuzzy-tolerance escalation; the implicit-zero <code>splitTolerance</code> default moved from 0.0 to 10^{-4} under <code>occ/ocp</code> (OCCT 7.9.3 is more sensitive to near-tangency); <code>_repair_non_manifold_solid</code> now validates its own reconstruction (validity + volume conservation) before trusting it.	Decomposition silently returning 1 piece instead of 2 or 3; fragments with inflated volume.
Surface classification	<code>is_same_plane_surface</code> compared two planes' offsets with anti-parallel axes as <code>d1 == d2</code> instead of <code>d1 == -d2</code> ; <code>convex_planes</code> had a dead sign test (<code>cross.dot(ref)</code> always 0) and did not normalise <code>atan2</code> to $[0, 2\pi)$; <code>multiplane()</code> compared an instance against a class (<code>Gclassify_curve(e)</code> is <code>GLine</code>), making <code>MultiPlane</code> undetectable; <code>is_closed_cylinder_cone</code> accepted a face that was not a closed 360° cylinder $\rightarrow$ a real topological invariant by winding (<code>_is_closed_by_winding</code>); <code>cyl_plane_region_conf</code> used the seed face for both ends of a split cylinder.	Faces classified into the wrong composite type; tallies of 0.0 and hundreds of σ deviations, all closed.
Tolerances	<code>min_area</code> was never wired into <code>order_plane_face</code> or the candidate generators; new <code>min_face_width</code> + <code>CharacteristicWidth</code> (characteristic width of a face via radii of gyration) to detect “narrow but large-enough-area” <i>slivers</i> , ported to all three engines; <code>Tolerances.scaled(volume)</code> scales <code>min_area/min_face_width</code> down for tiny decomposed fragments; <code>eligible_plane</code> ignored the user-configured tolerances.	Spurious sliver planes entering the cell definition $\rightarrow$ lost particles.
Residual boolean-cut slivers	<code>other_face_edge</code> with a <code>skip_slivers</code> mode to walk past sliver faces and find the real one; sliver filter in <code>merge_same_surface_faces</code> before the $O(n^2)$ adjacency walk; <code>sort_range</code> with a “no-progress” fallback for fragments joined only modulo 2π (was infinite recursion).	Can/RoundCorner detection failing or hanging on geometry with cut artefacts.
Cross-engine	The <code>ShapeUpgrade_UnifySameDomain</code> flag that triggers a native crash is <i>opposite</i> between OCP and <code>pythonocc-core</code> on the same OCCT 7.9.3 geometry; <code>get_join_cone_cyl</code> used <code>.Index</code> where a merged shell has <code>.Indexes</code> (<code>freecad</code> only); MKL's LAPACK conflicted with <code>occt/tbb</code> in the same process $\rightarrow$ switch to OpenBLAS in <code>pyoccenv</code> and <code>ocpenv</code> .	47 tests crashing on <code>freecad/occ</code> with an <code>AttributeError</code> ; native access violations.

Category	Representative examples	Impact
Non-geometric, found along the way	<code>generic_split</code> double-wrapped an already-wrapped <code>GSolid</code> → decomposition was silently a no-op; <code>check_intersection</code> with a <code>Gdistance</code> early-exit classified fully nested solids as disjoint → the enclosure void cell did not exclude the spheres inside it (<code>w_encl.stp</code> , tally 0.0); <code>expand_regions_to_integer</code> lacked a <code>bool</code> guard that all five cell writers needed and only MCNP had.	Correctness bugs that degraded decomposition or output invisibly.

8 Phased pyOCC migration

The pyOCC migration was tackled in phases, each with an explicit go/no-go criterion before writing code at scale.

1. **Phase 1 — Scaffolding and go/no-go test.** Engine selection via `GEOUNED_CAD_ENGINE`; `geo/__init__.py` imports from `_freecad_impl` or the new `_occ_impl`. Only the minimum was implemented to test `Gsplit` against `rev_pipe.stp`, already characterised as broken in FreeCAD. Result: `BOPAlgo_Splitter` (OCCT 7.9.3) returns **3 valid solids** on the first attempt with no repair, where FreeCAD returned 1 invalid solid. A strong signal to continue.
2. **Phase 2 — Full port.** All 5 surface descriptors and 4 curve descriptors, `Gclassify_surface/Gclassify_curve`, full `GEdge/GWire/GFace/GShell/GSolid` (including `find_interior_point`, `faces_sharing_edge`, `fix/refine`), all the `Gmake_*` constructors and the boolean operations. New test `tests/geo/test_occ_impl.py`: 39/39 against real geometry with known expected values.
3. **Phase 3 — Integration pipeline.** `tests/test_cadtocsg.py` passes 50/50 under `GEOUNED_CAD_ENGINE=occ`, identical to the FreeCAD baseline, after closing 6 real gaps (among them `Gload_step_labels` via `XCAF`, a periodicity bug in `get_join_cone_cyl` reachable under FreeCAD too, and the `GeomAPI_ProjectPointOnSurf` fallback).
4. **Phase 4 — Solidos/ corpus.** Differential scan of 98/99 files against a fresh FreeCAD baseline; 7 more gaps closed. `rev_pipe.stp` confirmed: FreeCAD fails the full decompose+convert path (`FuseEdges`), `occ` succeeds with `RoundC=2`, `MultiRoundC=1`.

8.1 Third engine: OCP (pybind11)

Added alongside FreeCAD and `pythonocc-core`, not as a replacement. OCP is a `pybind11` binding of OCCT that is more direct and more actively maintained than `pythonocc-core`'s SWIG. Go/no-go validation against `rev_pipe.stp` (2 pieces, volume conserved), full port structurally copied from `_occ_impl.py` and retargeted (with API differences confirmed live: `_s` suffix on static methods, no `.DownCast()`, swapped argument order in `ShapeUpgrade_UnifySameDomain`, ...). `tests/geo/test_ocr_impl.py` 39/39 on the first run; `tests/test_cadtocsg.py` 50/50; `tests/test_csgtocad.py` 2/2. Both pyOCC engines are developed in parallel: a fix in one is ported to the other.

The performance gap, reported honestly. On the motivating case (`hylife-v06.stp` solid 17), FreeCAD took 2.3s and `pythonocc-core` 19.2s. It was first attributed to the per-call cost of the SWIG binding (real: `pythonocc-core` exposes the raw OCCT API without the C++ convenience layer FreeCAD has). OCP on the same case: 18.4s — only ~4% faster, nowhere near the 2–6× of the point-conversion micro-benchmarks. The real cause turned out to be **something else**:

`splitTolerance` with an implicit value of 0.0 made OCCT 7.9.3 compute and then discard a degenerate 15-fragment split. With the default corrected to 10^{-4} under `occ/ocp`, the same case drops to **1.2s** — faster than FreeCAD.

9 GEORreverse migration

GEORreverse (CsgToCad) reconstructs CAD from CSG and historically depends on FreeCAD's `.FCStd` document format, not just its geometry kernel. It was initially permanently out of scope; on the current branch (`georeverse-migration`) it has received the same treatment as the forward pipeline.

- **Decoupling.** The GEORreverse core (`Objects.py`, `buildSolidCell.py`, `splitFunction.py`, `buildCAD.py`, `MCNPinput.py`) imports `geo` directly, like the forward pipeline. A `matrix_utils.py` module holds the numpy $\leftrightarrow$ native-matrix bridge. `BOPTools.SplitAPI.slice` $\rightarrow$ `Gsplit`; `FuseSolid` (triplicated byte-for-byte) $\rightarrow$ a single `fuse_solids` with `GSolid.refine()`'s volume-invariance check.
- **Engine-independent `export_cad`.** `export_cad(output_filename="", format="stp")` accepts a `str` or a list of formats, validates against the engine's supported formats and dispatches by `CAD_ENGINE`. Real behaviour change: `.FCStd` is no longer written unconditionally, only when `"fcstd"` is explicitly requested.
- **Full FreeCAD/pyOCC symmetry.** `GEORreverse/Modules/__init__.py` dispatches between `_freecad_impl.py` (export via a FreeCAD document, `makeTree`) and `_ocp_impl.py/_occ_impl.py` (real export via XCAF: `STEPCAFControl_Writer`, a `Universe` $\rightarrow$ `Material` $\rightarrow$ `Cell` hierarchy with names and nesting). The 6 exotic quadric surfaces were reorganised into a `geo_quadrics` package with the same dispatch pattern.
- **Per-material colour (`occ/ocp` engine).** Each solid in the exported STEP is coloured by its `MAT` value via XCAF: the same colour for the same material, a `tab10` palette up to 10 materials and golden-angle rotation beyond that, with no collisions. Colour on the FreeCAD engine is out of scope (its colour API lives in `Gui.ViewProvider`, with no headless path).
- **IGES dropped.** It was verified live that `IGESControl_Writer` does not preserve solids through IGES in this OCCT build (a solid comes back as trimmed planar faces with no shell topology). IGES import and export were abandoned.

Verification: `tests/test_csgtocad.py` is engine-aware and passes 2/2 under all three engines against real per-solid volume assertions ($\pm 10^{-6}$). Under `occ/ocp` there remain 2 pre-existing failures (GEORreverse produces 2/1 solids where 4/5 are expected in the `cylinder_box` STEP), confirmed not introduced by this migration and not tied to the `Gsplit` work.

10 CAD defect detection and repair

A new load-time layer (`Gload_and_process_step` $\rightarrow$ `Gcheck_and_repair`) that, for each solid: applies a `fix(1e-6)`, checks for known defects and, if any, tries to repair them with the most appropriate method; only if the repair fails is the user-chosen mode consulted. The `corrupted_solids` parameter has two modes, `"stop"` (default) and `"remove"`; the old `invalid_solids` parameter was merged into it.

Defect	Detection	Repair	Status
Split boundary ring (duplicated micro-trim / collapsed step, spherical variant)	<code>find_split_ring_faces</code> : a sliver-scale analytic curved face that also touches a pathologically short edge. <code>count_split_ring_pairs</code> counts pairs of near-coincident, concentric circular edges.	<code>Gcollapse_split_rings</code> (ocp/occ): removes the “riser” faces, re-sews the shell, rebuilds the solid. Acceptance by 3 checks: validity + $ \Delta V /V < 3 \times 10^{-4}$ + the ring pairs must decrease.	repaired <code>barrel</code> <code>bottom.stp</code> → tally 0.9997
Cylindrical collapsed step (beltline family)	The same split-ring detector fires; <code>near_surface_pair</code> identifies two cylinders of near-equal radius bridged by a sliver band.	<code>Gsliver_heal</code> (ocp/occ), version 0: removes the sliver faces and the smaller of the near pair, re-trims the freed quadric to its real radius, sews <i>last</i> . Gate: validity + $ \Delta V < 5 \times 10^{-4}$.	repaired <code>LR.stp</code> → tally 0.9989 (was 0.0 / 24 lost particles)
Phantom cut (two volumetric regions touching only along edges — the migration’s founding motivation)	<code>_separate_edge_joined_components</code> non-manifold edges (shared by $\neq 2$ faces) → a face-adjacency graph over manifold edges only → if there are ≥ 2 connected components and the non-manifold edges were the load-bearing connection.	<code>SolveIt</code> each component separately (no donor faces: a real phantom cut already carries each region’s closed boundary). Gate: ≥ 2 pieces, all valid, summed volume conserved to 10^{-6} .	resolved ocp/occ · <code>Piece_1+Tool_1</code> → 2 valid solids (was 1 invalid merged solid). FreeCAD cannot do it.
Malformed face wire (<code>UnorientableShape</code>)	<code>check_solid_defects</code> reports real invalid topology; <code>RepCheck_Analyzer</code> flags the face, no sliver edge.	None. <code>ShapeFix_Shape</code> , <code>ShapeFix_Face</code> , wire partitioning and rebuilding were all tried — they all fail or drift the volume $> 1.7\%$.	<i>no repair</i> <code>L4_support.stp</code> moved to <code>Solidos/Detected_corrupt</code> excluded from conversion

A related change: `check_solid_defects` no longer rejects a solid for a sliver edge on a *well-formed* face (normal `CharacteristicWidth`) — the decomposition pipeline already tolerates sliver edges on sound faces, and no repair removes such an edge without a sliver face to anchor on. The defect recipes are kept in a living catalogue (`reference_cad_defect_recipes.md`): observable symptom, a general detection signature, root cause, a repair if it is fast and reliable, and a minimal STEP fixture with dlsuned verification.

11 Current status and open work

11.1 Consolidated status

- All three engines pass their suites: `freecad` 158/158, `occ` 128/128, `ocp` 128/128 (+2 pre-existing GEORreverse failures under the pyOCC engines).
- Corpus `Solidos/test_models`: $93.6\% < 2\sigma$, $0\% > 3\sigma$, 0 lost particles across 143 dlsuned runs. `occ` and `ocp` are byte-for-byte equivalent over the whole corpus.

- `ocp` is the default engine when `GEOUNED_CAD_ENGINE` is unset.

11.2 Open

Item	Nature
Volume discrepancy in the GEORreverse round-trip on <code>hylife-v06.stp</code> cell 1 (reconstructed CAD $1.401\times$ vs <code>dlsuned</code> tally $1.119\times$).	GEORreverse, not traced to the bottom.
<code>test_csgtocad.py</code> 2 failures under <code>occ/ocp</code> (2/1 solids where 4/5 are expected).	GEORreverse, pre-existing, narrowed to <code>Objects.py::CellObj.buildShape</code> .
<code>AdjacentMultiplanePlanes</code> needs extending from <code>RevCC</code> to <code>MultiRoundCorner</code> .	Coverage improvement, open since 2026-08-21.
The 6 exotic quadrics (<code>Gmake_elliptic_cone</code> , <code>Gmake_hyperboloid</code> , ...) are stubs in GEORreverse's py-OCC backends.	Port pending; no fixture exercises them yet.
<code>Big_complex_cell/modelcell_cut1.stp</code> and <code>modelCell_670000.stp</code> lose particles.	Not traced; excluded from corpus scans by convention.
<code>hylife-v06.stp</code> full 372-solid round-trip: slow decomposition in <code>meta_list</code> [45] (duplicated face fragments, $O(n^2)$ cost in <code>same_faces</code>).	Root cause identified, not optimised, deprioritised by the user.
<code>ConeSphere.stp</code> : native crash (access violation) under <code>occ/ocp</code> in <code>ShapeUpgrade_UnifySameDomain</code> ; translates clean under <code>freecad</code> .	<i>accepted limitation</i> No OCCT knob avoids it; documented in the docstrings.
Reorganisation and dedup of the <code>Solidos</code> / fixture tree.	Partial (<code>test_models/</code> + a few <code>duplicates_removed/</code>).

Explicit user priority: finish cleaning up the known bugs in the forward pipeline (GEOUNED) before resuming the GEORreverse items.

12 Appendix: environment and tooling

conda env	Binding	OCCT · Python	Use
(default python)	FreeCAD 1.1.1	7.8.1 · 3.11	<code>GEOUNED_CAD_ENGINE=freecad</code>
<code>pyoccenv</code>	<code>pythonocc-core</code> (SWIG)	7.9.3 · 3.12	<code>GEOUNED_CAD_ENGINE=occ</code>
<code>ocpenv</code>	OCP (pybind11)	7.9.3 · 3.12	<code>GEOUNED_CAD_ENGINE=ocp</code> (default)

- **FreeCAD and pyOCC cannot coexist in one process** (Module use of `python311.dll` conflicts): FreeCAD ships Python 3.11, the pyOCC environments use 3.12. Any pyOCC-side test uses that environment's `python.exe` and never mixes in the same invocation.
- The pyOCC environments needed the BLAS/LAPACK provider switched from MKL to OpenBLAS (`conda install -c conda-forge -override-channels "libblas==openblas" ...`): MKL's LAPACK conflicts with `occt/tbb` in the same process and causes an access violation in `numpy.linalg`.
- The MCNP volume check runs on `dlsuned` (an in-house MCNP variant) through WSL. The full pipeline, scripts and results are archived under WSL (Ubuntu-22.04) at `~/work/taller/SolidTestMCNP/scripts/` (parallel conversion and `dlsuned` per

engine: `run_all_conversions*_parallel.py`, `run_test_models*_disuned.sh`,
 `analyze_test_models_*.py`).

- Working-tree convention: `Solidos/` holds STP only; `Solidos/test_models/` is the curated regression set; `../calculos_CLAUDE/` holds every calculation run; `Solidos/BadCAD_decomposition/`, `Solidos/Detected_corrupted/`, `Solidos/no_convierte/`, `Solidos/convierte_bad_volume/` are triage folders.

Report synthesised from the GEOUNED project working log CLAUDE.md. Branch georeverse-migration. Date: 2026-09-06.

Project code-style preference: communication is in Spanish; all code, comments, docstrings and identifier names are in English.